iFive AutoQA X

Enterprise Autonomous AI Testing Platform

Complete Project Concept, Architecture, End-to-End Flow, Phase Plan, Requirements and Team-Lead Demo Strategy

Temporary local proof-of-concept first - company integration only after approval

Prepared for internal review

17 July 2026

CONFIDENTIALITY AND SAFETY NOTE

This document describes a safe proof-of-concept. The first implementation must use a mock task portal, a demo product website, demo credentials and isolated data. No company task database, production URL, real employee credentials or customer data should be connected until written team-lead and security approval is obtained.

Document Control

Item	Value
Document title	iFive AutoQA X - Complete Project Concept and Implementation Plan
Version	1.0 - Proof-of-Concept Blueprint
Purpose	Define the complete temporary PoC, end-to-end workflow, implementation phases and later company integration path
Primary users	Superadmin, Tester, Developer, Delivery Team and Project Lead
Initial deployment	Local isolated environment
Later deployment	Approved company development environment, then controlled cloud/server deployment
Primary decision	Manual tester approval is mandatory before every task-based execution
Retention baseline	30 days for screenshots, logs, videos, traces and reports

Core product statement

A developer moves an assigned task to Testing L2. AutoQA imports that exact task, understands the required workflow, generates reviewable test cases, waits for tester approval, executes the approved flow in a real Playwright browser, captures evidence and returns PASS, FAIL or BLOCKED. PASS goes to delivery after human confirmation; FAIL returns to the developer as Rework.

How to Use This Document

- Use Sections 1-6 to explain the idea and business value to a team lead.
- Use Sections 7-16 as the functional and technical implementation specification.
- Use Sections 17-19 as the phased development plan and demo acceptance checklist.
- Use the appendices as sample tasks, test plans, action schemas and report templates.

Contents

- 1. Executive Summary
- 2. Business Problem and Product Vision
- 3. Final Scope and Locked Requirements
- 4. Existing Company Workflow Reference
- 5. Proposed Temporary Proof-of-Concept
- 6. End-to-End Business and System Flow
- 7. Roles, Permissions and Status Lifecycle
- 8. Functional Module Specification
- 9. AI Planning and Structured Test Generation
- 10. Playwright Execution and Locator Model
- 11. Validation, Evidence and Result Decision
- 12. Defects, Reports, Dashboard and Notifications
- 13. Security, Data Safety and Production Controls
- 14. Architecture, Data Model and Recommended Folder Structure
- 15. Technology Stack and Installation Matrix
- 16. APIs and Integration Contracts
- 17. Complete Phase Plan
- 18. MVP Demo Scenarios and Acceptance Criteria
- 19. Team-Lead Approval and Real Company Integration

- 20. Current Project State and Remaining Work
- 21. Risks, Mitigations and Governance
- Appendices

1. Executive Summary

iFive AutoQA X is an enterprise autonomous QA platform designed to test a web application like a senior QA engineer while keeping critical decisions under human control. The system is task-driven: it does not randomly test every page. It reads a developer task, identifies the exact feature or fix, generates a reviewable test plan, executes only after approval and produces evidence-based results.

- First build a safe local proof-of-concept with no company connection.
- Use the developer task as the exact test scope; related regression remains optional.
- Require manual approval before execution and final delivery confirmation after PASS.
- Use real Playwright actions and genuine evidence; never accept dummy success.

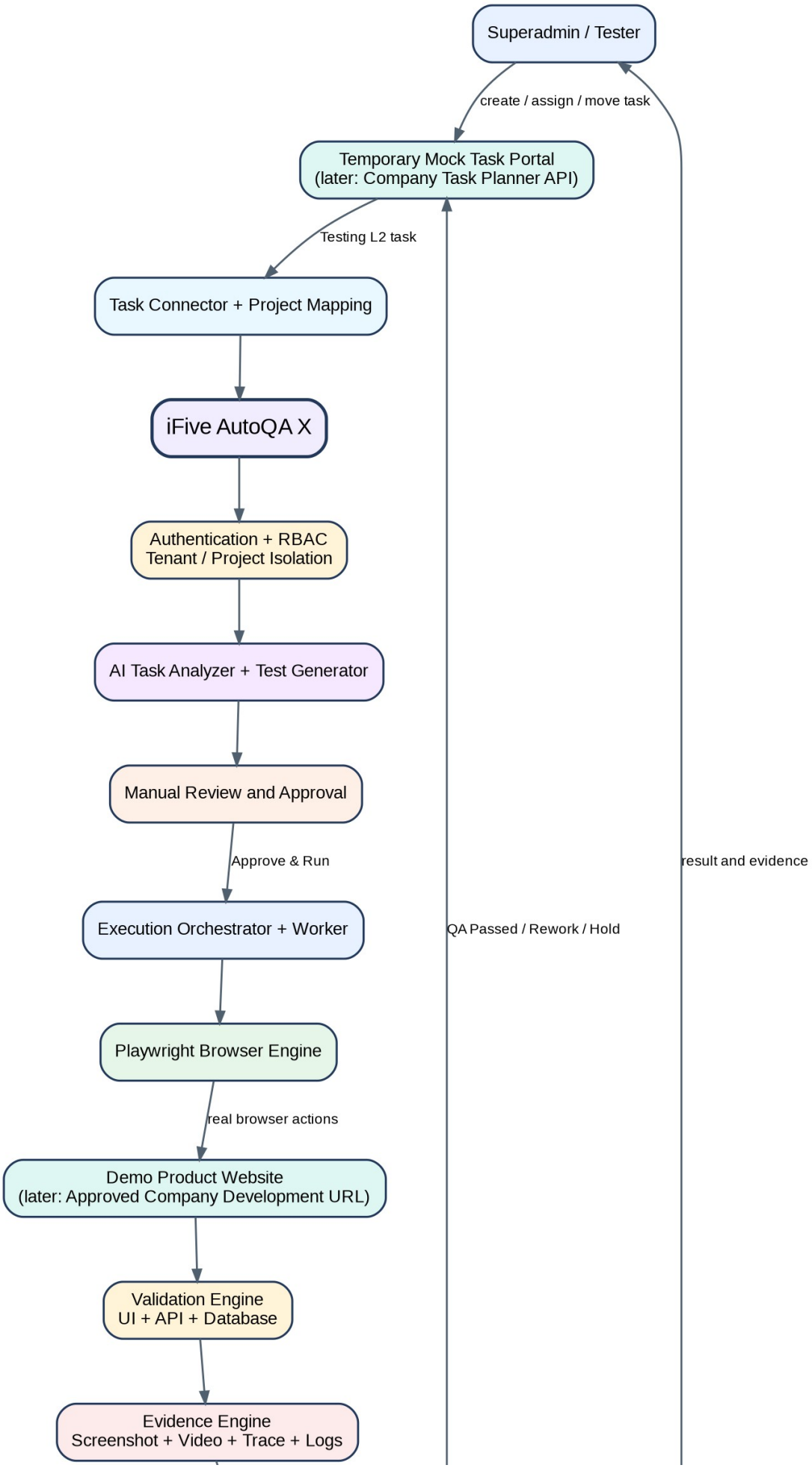

Superadmin / Tester
create / assign / move task
Temporary Mock Task Portal
(later: Company Task Planner API)
Testing L2 task
Task Connector + Project Mapping
iFive AutoQA X
Authentication + RBAC
Tenant / Project Isolation
AI Task Analyzer + Test Generator
Manual Review and Approval
Approve & Run
Execution Orchestrator + Worker
Playwright Browser Engine
real browser actions
Demo Product Website
(later: Approved Company Development URL)
Validation Engine
UI + API + Database
Evidence Engine
Screenshot + Video + Trace + Logs
QA Passed / Rework / Hold
result and evidence

Figure 1 - Master architecture and information flow

Recommended delivery strategy

1. Build a local Mock Task Portal that reproduces the company Testing L2 workflow.
2. Build a safe Demo Product Website with real forms, tables and configurable defects.
3. Connect AutoQA only to these local systems and demonstrate the full task-to-result workflow.
4. Obtain team-lead, security and access approval.
5. Replace the mock connector with an approved read-only company Task Planner API and map only approved development URLs.
6. Keep production testing limited, non-destructive and separately approved.

> **Most important design rule**
>
> AI may analyze a task and generate test suggestions, but a trusted deterministic executor performs the browser actions and validation. No AI-generated action can run until it passes schema validation and a Superadmin or Tester approves it.

Target outcome

```
Developer completes assigned task
        -> Task status becomes Testing L2
        -> AutoQA imports the task
        -> AI creates task-specific test plan
        -> Tester reviews and approves
        -> Playwright executes the real workflow
        -> UI / API / Database are validated where configured
        -> Screenshots / Video / Trace / Logs are captured
        -> PASS / FAIL / BLOCKED is produced
        -> PASS: Delivery Ready after confirmation
        -> FAIL: Rework with evidence-backed defect
        -> BLOCKED: Need Clarity or Hold
```

2. Business Problem and Product Vision

2.1 Current manual process

In the current process, a developer receives a task, makes the change and sends it to Testing L2. A tester manually reads the task, opens the correct product, logs in, finds the relevant menu, performs the workflow, checks the expected result, captures evidence and decides whether the task should go to Delivery Completed or Rework. This process is slow, repetitive and dependent on individual tester knowledge.

2.2 Target process

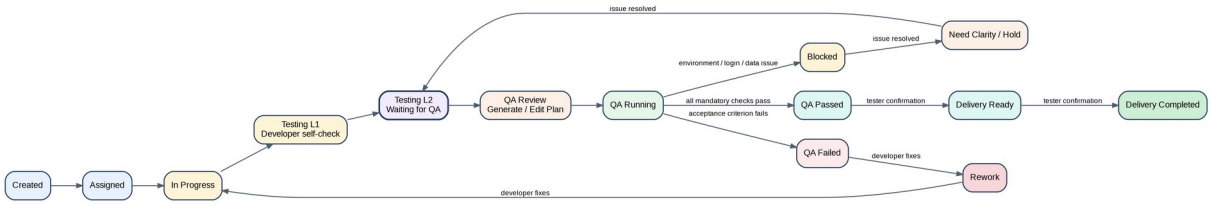


Figure 2 - Complete task status lifecycle

2.3 Product vision

The final platform should accept any approved company product URL and any task-specific flow. It should understand the task, identify the affected module, generate only relevant test cases, execute them in the browser and return a professional result. The first version will prove this capability using a temporary task portal and demo product; the same connector interfaces will later support the real company systems.

2.4 What "full web check" means

Testing area	Examples	First PoC status
Functional	Login, forms, create/update, search, workflow, validation	Core task flow required
UI	Visibility, alignment, responsive layout, broken controls	Basic checks in PoC; advanced visual comparison later
API	Request trigger, status code, response fields, response time	Task-related APIs after core UI flow
Database	UI value = API value = stored value	Read-only configured validation after core flow
Accessibility	Keyboard, labels, ARIA, contrast	Later advanced phase
Performance	Page load, API timing, Web Vitals	Later advanced phase
Security	Auth, role access, session and safe headers	Platform security now; target checks only with approval
Evidence	Before/after/failure/success screenshots, trace, video, console/network	Required for the PoC

3. Final Scope and Locked Requirements

3.1 Locked business requirements

Requirement	Decision
Target selection	The user provides an approved URL and the task path/flow; AutoQA tests that exact scope.
Module selection	No fixed first module. The selected developer task decides the module and feature.
Task source	Temporary Mock Task Portal first; real company internal Task Planner only after permission.
Start condition	Task enters Testing L2, but execution starts only after manual Superadmin/Tester approval.
Authentication	Username and password; first-time OTP is a manual checkpoint.
Test accounts	Support isolated profiles for two testers and one Superadmin.
Test data	Use existing safe data or generate unique test data according to the approved plan.
Primary scope	The exact developer fix or feature is always the primary test scope.
Regression	Related regression is suggested but disabled by default until the tester enables it.
Expected result	Use task-provided expected result; AI may draft it when missing, but tester approves.
Testing depth	Positive and negative tests are required; boundary tests require explicit approval.
Validation	UI and API validation required; database validation when read-only access is configured.
Evidence	Before, after, failure and success screenshots; trace and video for

Requirement	Decision
	main runs.
Result	PASS, FAIL or BLOCKED. AutoQA must never assume success.
Defects	Create inside AutoQA with all evidence fields; AI may suggest severity.
Developer suggestion	Do not display AI code-fix suggestions in the first version.
Reports	Dashboard summary for all runs; detailed PDF for failed runs.
Notifications	Dashboard and email to Superadmin and assigned tester.
Browsers	Chromium/Chrome, Edge and Firefox; start with Chromium in the PoC.
Retention	30 days by default.
Isolation	Multiple companies/projects with strict data isolation.
Deployment	Local now; approved server/cloud deployment later.

3.2 First-version must-have outcome

MVP statement

Give the system a task and an approved demo URL. It must understand the task, generate a test plan, wait for approval, perform the workflow in a real browser, capture evidence and produce a trustworthy result and failure PDF.

3.3 Explicitly postponed

- Direct connection to the real company Task Planner before approval.
- Production data mutation, destructive tests or security payload testing without separate approval.
- Full OCR, deep security scanning, distributed workers, large-scale parallel execution and Safari/WebKit.
- Automatic delivery completion without tester confirmation.
- AI-generated developer code changes or automatic code-fix suggestions.

4. Existing Company Workflow Reference

The supplied screenshots show the current ERP Project Task Planner and an example product module. They are included as reference material so that the temporary PoC can reproduce the same operational idea without touching the company systems.

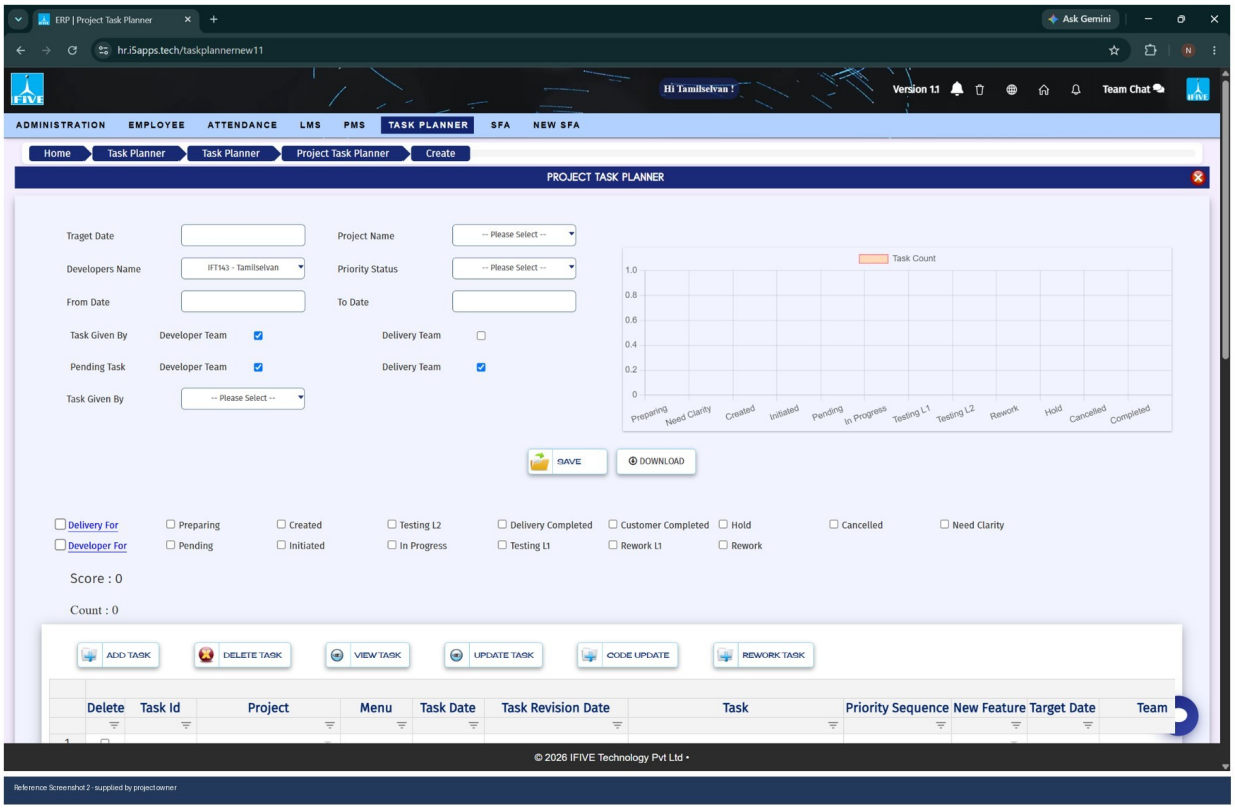


Figure 3 - Existing Project Task Planner: filters, project fields and task status controls

Reference only. The PoC will reproduce the workflow using local demo data.

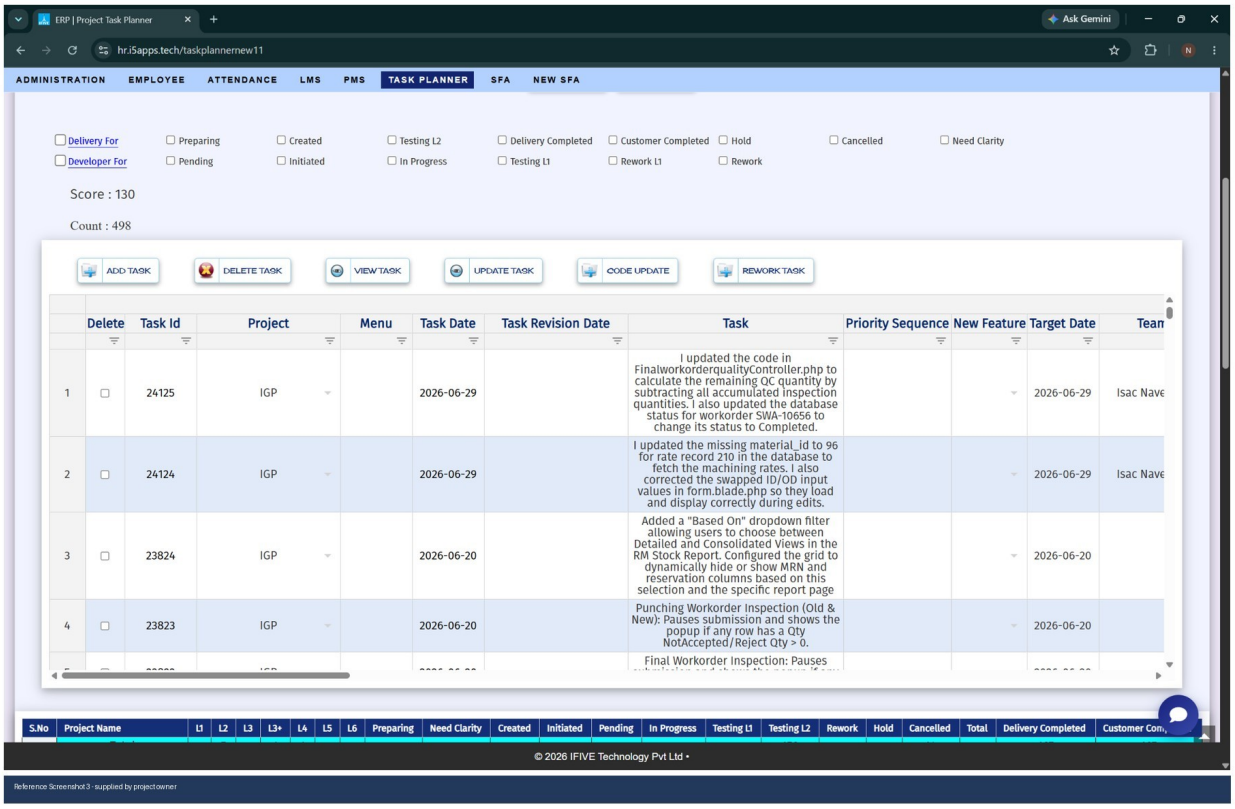


Figure 4 - Existing task list showing task ID, project, date, description and team

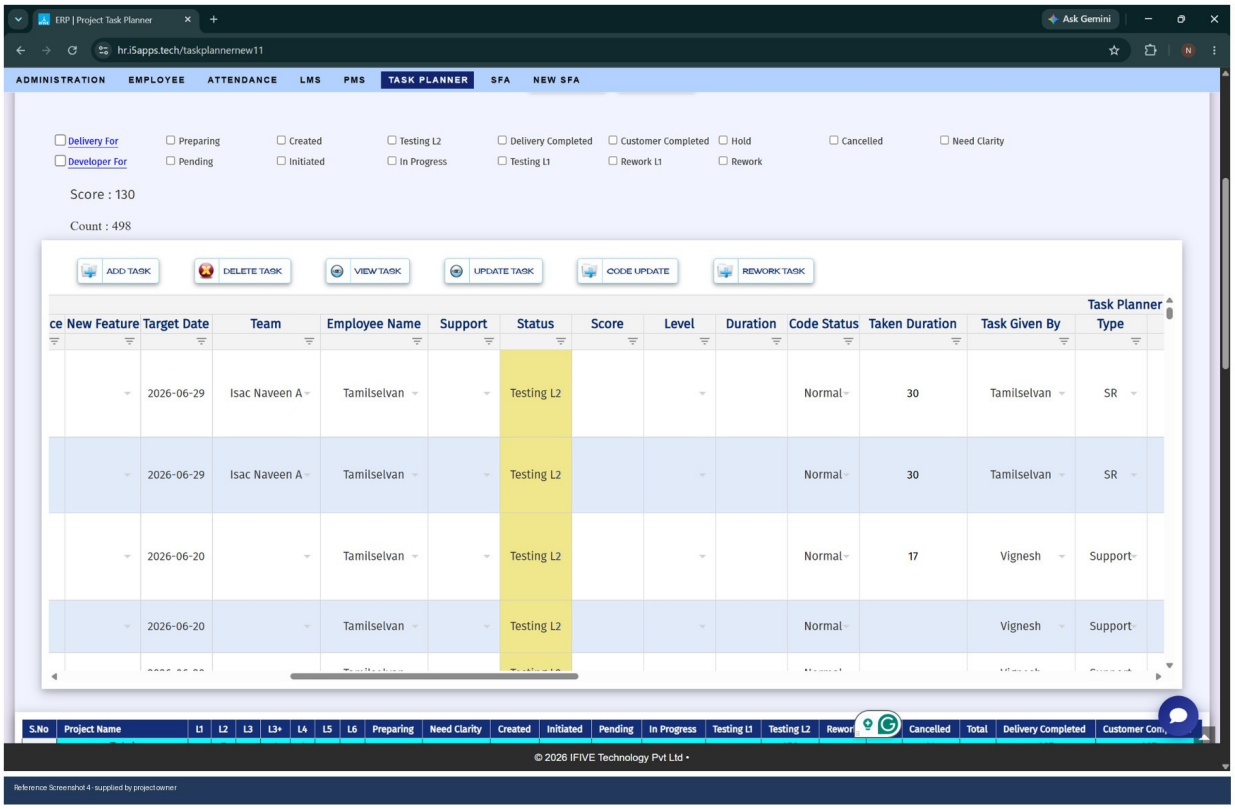


Figure 5 - Existing task list showing employee, status, duration, owner and type

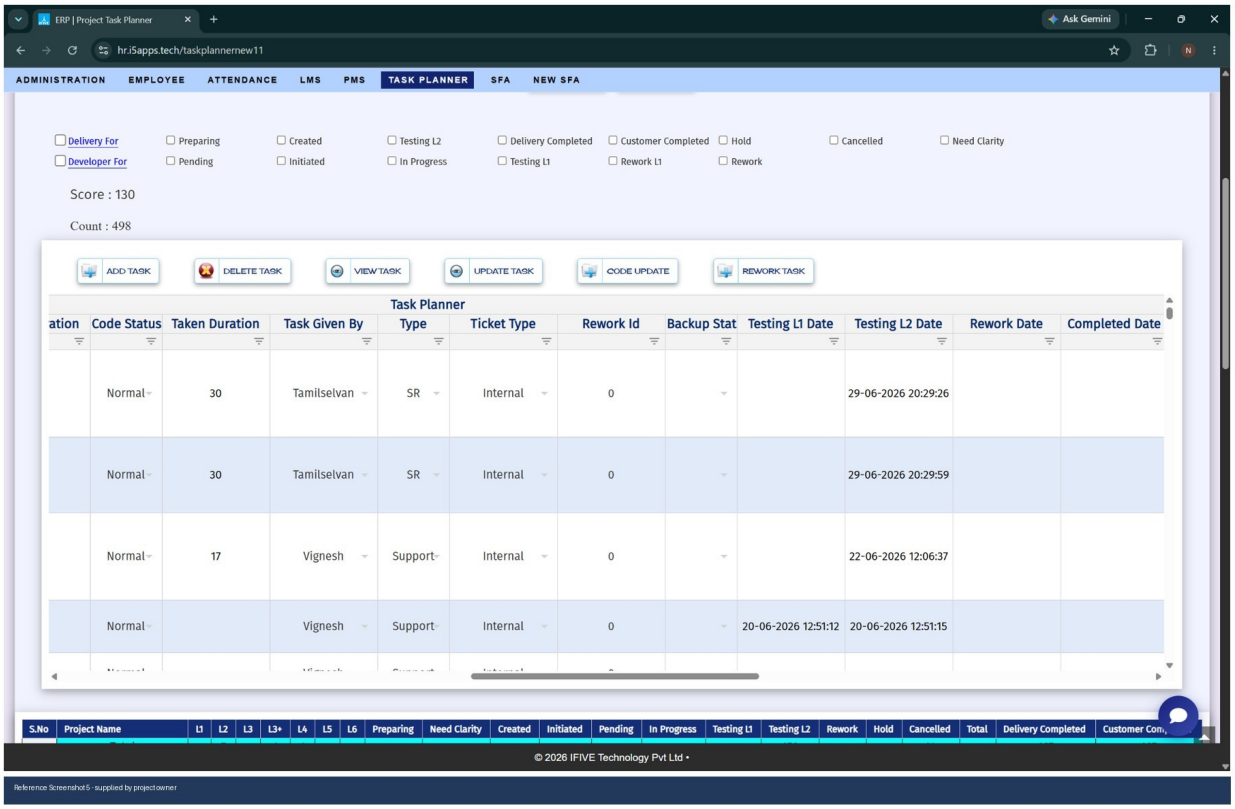


Figure 6 - Existing task lifecycle timestamps including Testing L1 and Testing L2 dates

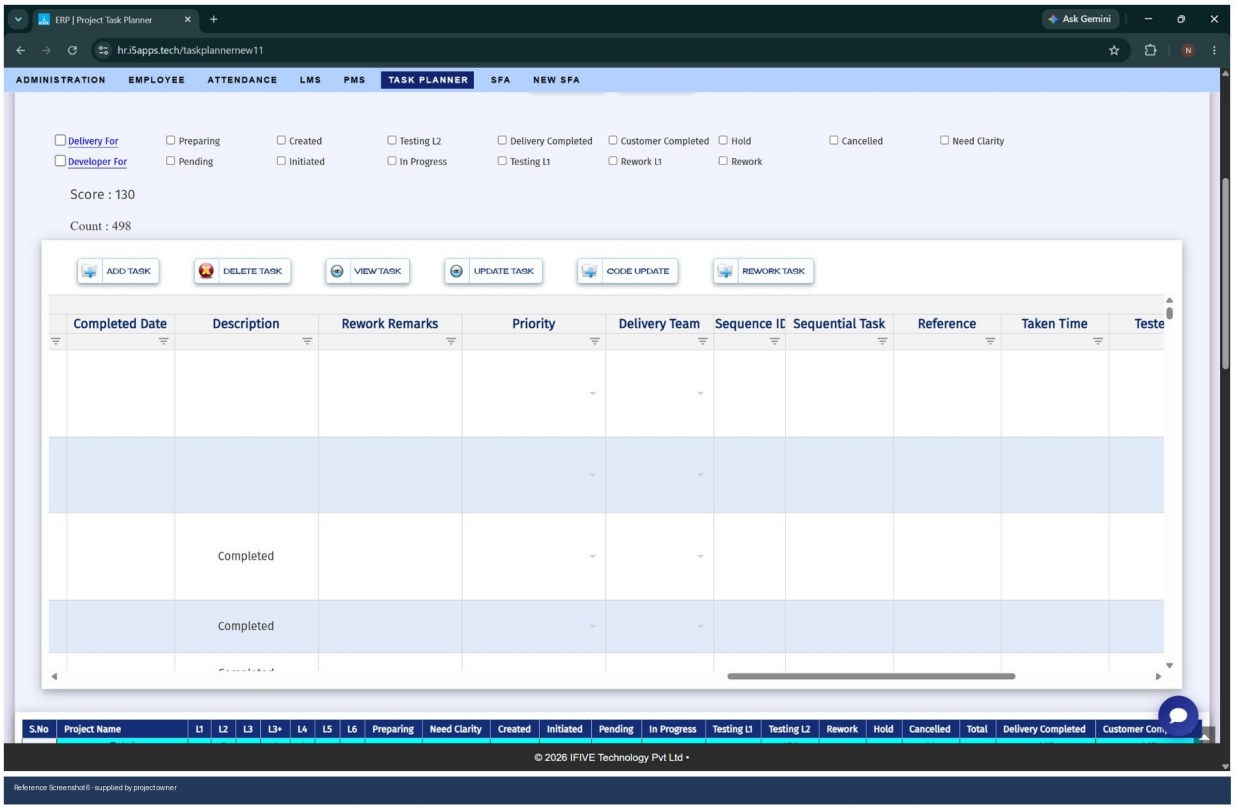


Figure 7 - Existing task details including description, rework remarks, priority and delivery fields

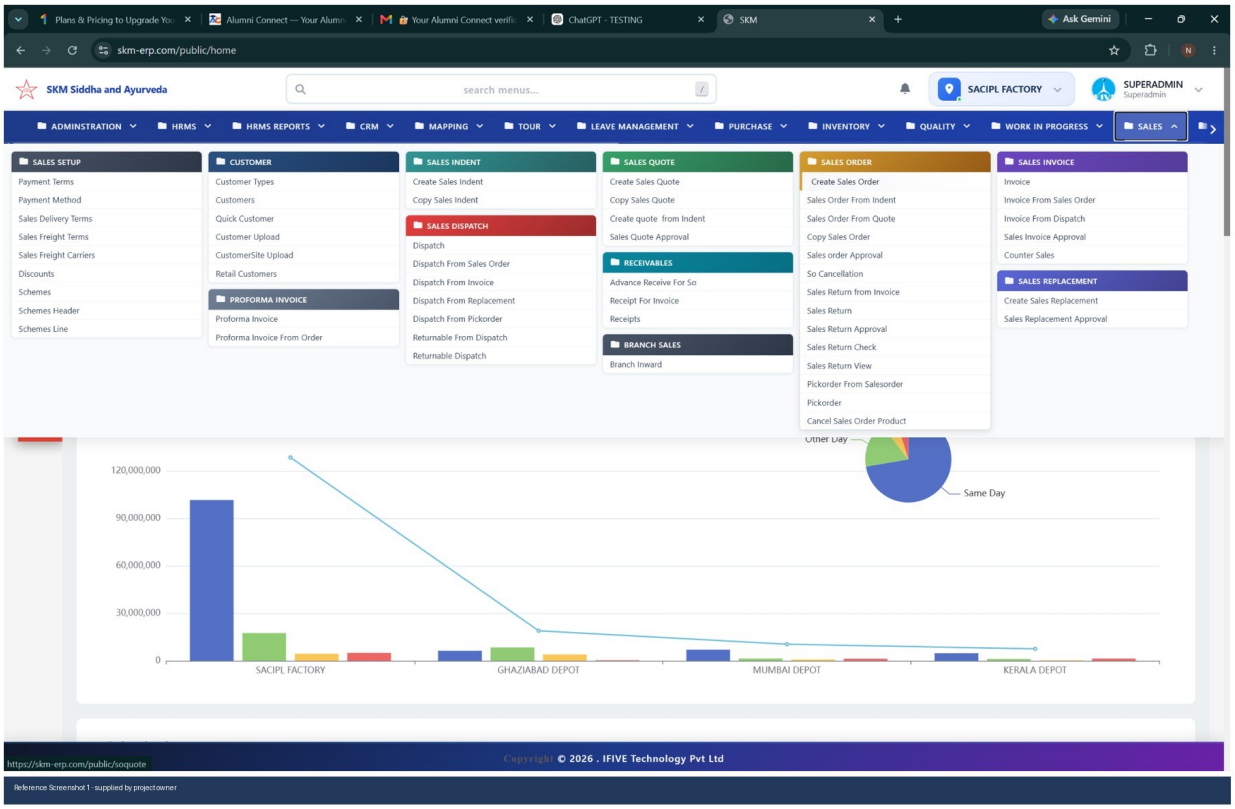


Figure 8 - Example ERP Sales module with many menus and transaction flows

This demonstrates why the platform must be task-driven and path-aware instead of randomly crawling every transaction.

4.1 Field mapping from the current Task Planner

Current field	AutoQA purpose
Task ID	Unique external reference and duplicate prevention key

Current field	AutoQA purpose
Project	Map to the correct product configuration and approved URL
Menu / Module	Guide the affected application area
Task / Description	Understand the developer change and expected behavior
Employee Name	Identify assigned developer
Task Given By	Identify reporter or task owner
Target Date / Priority	Prioritize the QA queue
Status	Import only Testing L2 tasks
Testing L2 Date	Measure queue and turnaround time
Rework Remarks	Use previous failure context for retesting
Ticket Type / New Feature	Classify support, fix, service request or new feature

5. Proposed Temporary Proof-of-Concept

5.1 Why temporary systems are required

The safest first step is to prove the complete concept locally. Connecting directly to the real Task Planner, real product login or production database before approval creates security, privacy and operational risk. The temporary PoC isolates those risks while proving the difficult parts: task interpretation, approval, browser execution, evidence and status decisions.

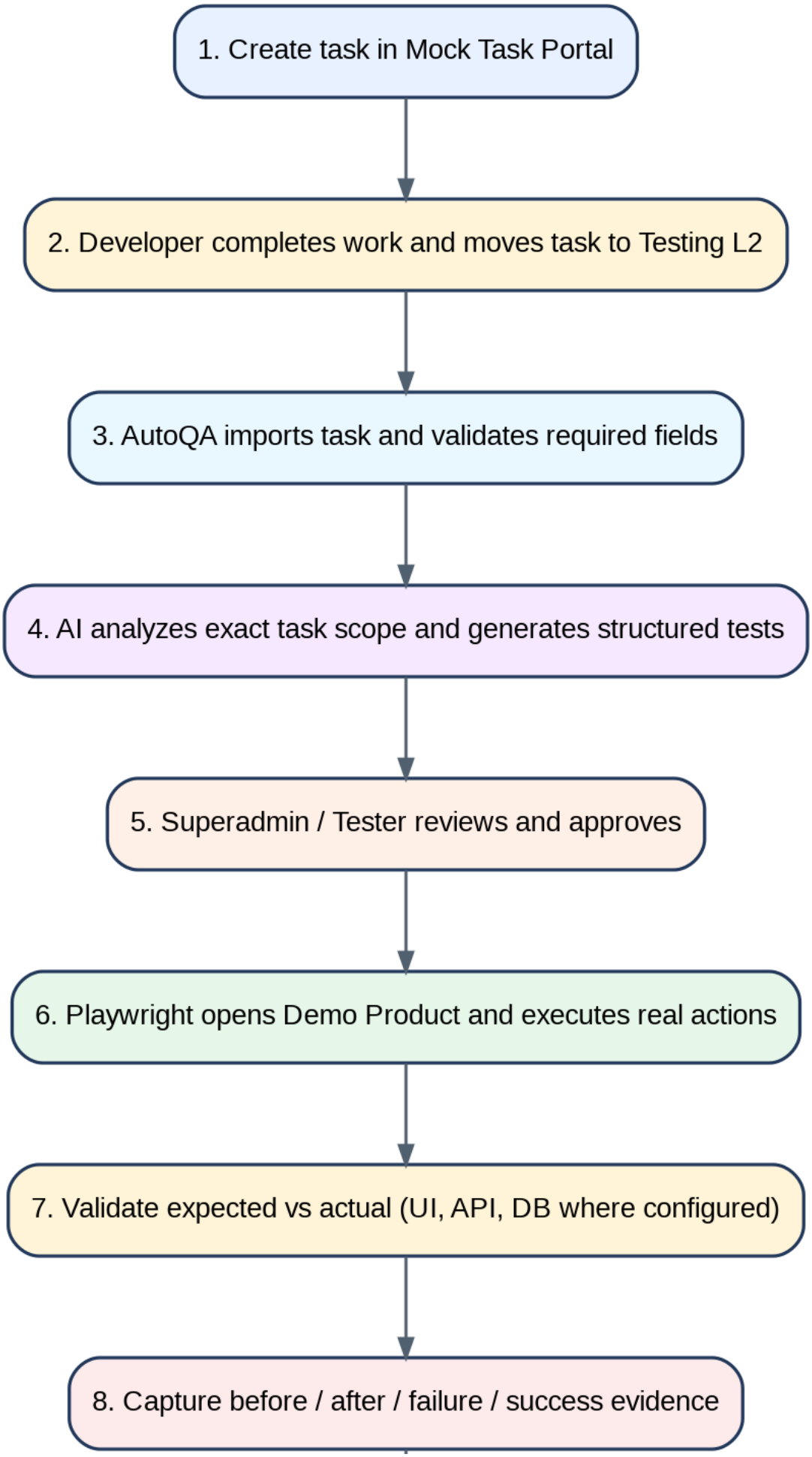

1. Create task in Mock Task Portal
2. Developer completes work and moves task to Testing L2
3. AutoQA imports task and validates required fields
4. AI analyzes exact task scope and generates structured tests
5. Superadmin / Tester reviews and approves
6. Playwright opens Demo Product and executes real actions
7. Validate expected vs actual (UI, API, DB where configured)
8. Capture before / after / failure / success evidence

Figure 9 - Temporary PoC end-to-end workflow

5.2 Temporary system components

Component	Purpose	Later replacement
Mock Task Portal	Create tasks, assign developers, move to Testing L2 and receive QA status	Approved company Task Planner API
Demo Product Website	Safe ERP-like product for real browser testing	Approved company development/staging URL
Demo identity profiles	Superadmin, two testers, developer and delivery users	Approved company test accounts
Local database	Store tasks, plans, runs, artifacts and defects	Approved PostgreSQL or company deployment database
Local email sink	Preview notifications without sending externally	Approved SMTP service
Local AI or configured LLM	Analyze task and generate structured test plans	Approved enterprise AI provider

5.3 Demo Product modules

- Login and Dashboard
- Employee form with textbox, dropdown, required validation and result table
- Customer and Product forms
- Sales Order form with quantity, rate, tax and discount calculation
- Reports page with filter and download button
- Configurable intentional bugs so PASS and FAIL demonstrations can be repeated safely

5.4 Demo workflow example

```
Task: DEMO-001 - Employee form Save button
Project: Demo ERP
Target: http://localhost:<demo-port>/employees/create
Expected: Employee is created and displayed in the table

Developer moves task to Testing L2
-> AutoQA imports the task
-> AI generates primary and negative cases
-> Tester approves the primary case
-> Playwright logs in and opens the form
-> FILL name, SELECT department, CLICK Save
-> ASSERT success message and table row
-> Capture screenshot, video and trace
-> PASS -> QA Passed -> tester confirms Delivery Ready
```

6. End-to-End Business and System Flow

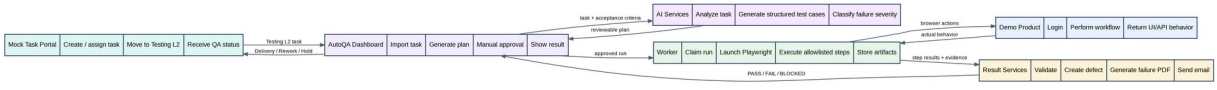


Figure 10 - Sequence of interactions across the PoC systems

6.1 Complete business flow

Step	Description
Task creation	Superadmin creates a task with project, title, description, target date, developer, URL/path, expected result and acceptance criteria.
Development	Developer accepts the task, changes status to In Progress, completes the change and records development notes.
Testing handoff	Developer completes self-check and moves the task to Testing L2.
Import	AutoQA connector imports the Testing L2 task and stores the source task ID and revision.
Readiness check	AutoQA validates project mapping, approved URL, required credentials, test data and expected result.
AI analysis	AI identifies the exact feature, risk, primary scope, negative scope, suggested data and missing information.
Plan generation	AI creates structured allowlisted test steps. Unsupported actions are rejected.
Human review	Superadmin or Tester edits, disables, enables and approves the test plan and selected environment.
Execution	The orchestrator queues the run. A worker launches Playwright and executes the approved test case.
Validation	The system compares expected and actual UI behavior, related API response and database result where configured.
Evidence	Before, after, success and failure screenshots plus trace, video, console and network logs are stored.
Decision	The result engine returns PASS, FAIL or BLOCKED using deterministic rules.
Defect/report	FAIL creates an AutoQA defect and detailed PDF; PASS creates a dashboard summary; BLOCKED records the blocking cause.
Status sync	PASS becomes QA Passed and waits for tester confirmation; FAIL becomes Rework; BLOCKED becomes Need Clarity or Hold.

6.2 Non-negotiable control gates

Gate	Condition	Failure behavior
Task completeness	Project, scope, expected result and target mapping exist	Need Clarity
Target approval	Domain and route are approved	Execution blocked
Test-plan approval	Tester/Superadmin approved a frozen version	Execution blocked
Credential readiness	Safe test profile exists; OTP checkpoint available	Blocked
Action allowlist	All generated actions are supported and schema-valid	Plan rejected
Evidence integrity	Required artifacts have valid size and hash	Run cannot be marked PASS
Final delivery	Tester confirms QA Passed result	Remain QA Passed; do not auto-complete delivery

7. Roles, Permissions and Status Lifecycle

7.1 Roles

Role	Primary permissions
Superadmin	Manage tenants/projects, configure URLs and credentials, approve tests, run tests, confirm delivery, view all reports and audit logs
Tester	Review imported tasks, edit plans, approve allowed runs, handle OTP, inspect evidence, confirm PASS/FAIL/BLOCKED
Developer	View assigned tasks, add development notes, move tasks through development statuses, receive rework defects
Delivery Team	View delivery-ready tasks and mark delivery completion according to company process
Viewer/Auditor	Read-only access to approved project reports and audit history
Worker service	Claim approved jobs and execute only within the assigned project and target policy

7.2 RACI matrix

Activity	Superadmin	Tester	Developer	Delivery	AutoQA
Create/assign task	A/R	C	I	I	-
Move to Testing L2	I	I	A/R	I	-
Generate plan	I	C	I	I	R
Approve test run	A	R	I	I	-
Execute browser flow	I	I	I	I	R
Classify PASS/FAIL/BLOCKED	A	R	I	I	R (rule-based)
Create defect	I	A	I	I	R
Confirm delivery readiness	A	R	I	I	-
Delivery complete	I	I	I	A/R	-

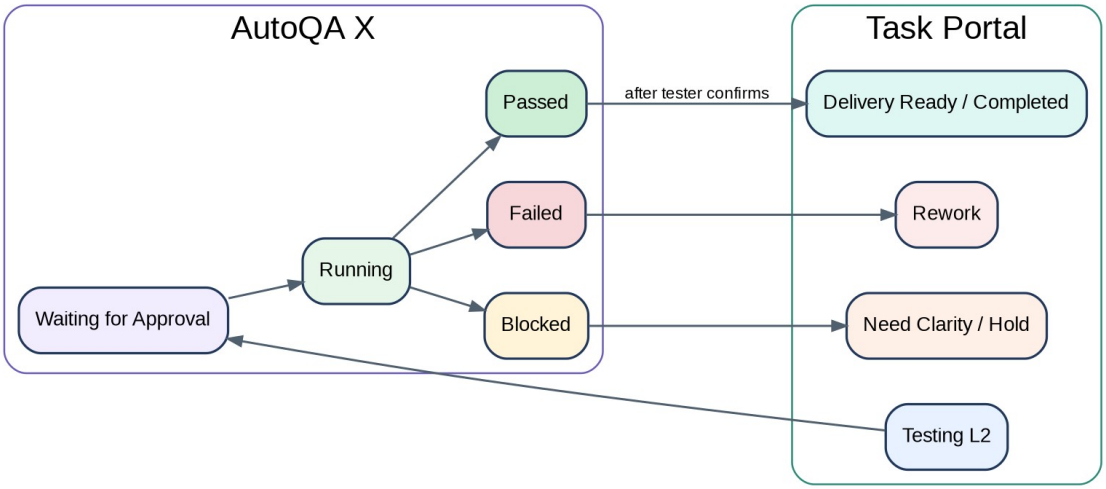


Figure 11 - AutoQA result to Task Portal status mapping

8. Functional Module Specification

8.1 Mock Task Portal

Local simulation of the company Task Planner. It must support login, task creation, assignment, task details, developer notes, status transitions, Testing L2 queue, rework and delivery views.

8.2 Demo Product Website

A safe ERP-like test target with login, forms, dynamic elements, dropdowns, tables, calculations and intentionally configurable bugs.

8.3 Task Connector

Fetch Testing L2 tasks, validate fields, prevent duplicates, store source revision and synchronize result statuses.

8.4 Project and Environment Mapping

Map project name to approved development/staging URL, allowed domains/paths, encrypted credential profile, API specification, read-only database profile, browser and cleanup policy.

8.5 Task Review Workspace

Display imported task, developer notes, AI understanding, missing information, planned test data, environment and approval controls.

8.6 AI Task Analyzer

Classify module, feature, risk, primary scope, negative scope, optional regression, required data and missing information.

8.7 AI Test Generator

Produce schema-valid structured tests using allowlisted actions and locators. Generated tests remain Draft until approval.

8.8 Approval Workflow

Require a Superadmin/Tester review, freeze the approved plan version and record actor, timestamp, environment and browser.

8.9 Execution Orchestrator

Queue jobs, atomically claim work, enforce timeout, retry policy, cancellation, worker heartbeat, stale recovery and browser cleanup.

8.10 Playwright Engine

Perform real browser actions, maintain same-origin policy, capture evidence and report actual failures instead of false passes.

8.11 Authentication and OTP

Use isolated browser contexts and encrypted credentials. Pause for first-time OTP and never store OTP values.

8.12 Validation Engine

Apply deterministic DOM, UI state, API, database, console and network checks.

8.13 Evidence Engine

Capture and hash screenshots, video, trace and logs; apply project authorization and 30-day retention.

8.14 Result Engine

Return PASS, FAIL or BLOCKED using defined decision rules.

8.15 Defect Manager

Create evidence-backed internal defects, support rework/retest history and allow severity override.

8.16 Report Generator

Dashboard summary for all runs and detailed PDF for failure results.

8.17 Notifications

Show dashboard alerts and send secure email notifications with links, not raw sensitive artifacts.

8.18 Audit and Governance

Record configuration changes, approvals, executions, downloads, status sync and access decisions.

8.19 Required task fields

Field	Required	Reason
Source Task ID	Yes	Traceability and duplicate prevention
Project Name	Yes	Project mapping
Task Title	Yes	Feature identification
Task Description	Yes	Understand change
Module/Menu	Recommended	Navigation hint
Application URL or project mapping	Yes	Safe target
Page path	Recommended	Exact starting route
Expected Result	Yes or AI draft + approval	Pass condition
Acceptance Criteria	Yes	Mandatory validations
Assigned Developer	Yes	Rework routing
Assigned Tester	Recommended	Approval routing
Target Date/Priority	Yes	Queue priority
Developer Notes	Yes at Testing L2	Changed behavior and known limits
Test Data	Recommended	Repeatability
Affected API/DB	Optional	Deep validation
Environment	Yes	Development/staging/production policy
Status	Yes	Import only Testing L2

9. AI Planning and Structured Test Generation



Figure 12 - AI analysis, approval and trusted execution pipeline

9.1 AI responsibilities

- Summarize the developer task in QA language.
- Identify module, feature, risk and exact primary test scope.
- Detect missing information and request clarification instead of guessing.
- Generate positive, negative and optional related regression scenarios.
- Suggest safe test data and responsive resolutions according to the task.
- Suggest failure severity based on business impact; tester can override.

9.2 AI restrictions

- AI must not directly control Playwright or generate arbitrary executable code.
- AI must not mark a run PASS. Deterministic validators decide the result.
- AI must not navigate outside approved domains or bypass authentication controls.
- AI must not generate destructive production tests without explicit permission.
- AI code-fix suggestions are not shown to developers in the first version.

9.3 Analyzer output schema example

```
{
  "task_id": "DEMO-001",
  "module": "Employee",
  "feature": "Create Employee",
  "risk": "medium",
  "primary_scope": [
    "open employee form",
    "create employee",
    "verify success message",
    "verify employee row"
  ],
  "negative_scope": ["submit without required fields"],
  "regression_scope": ["employee search"],
  "required_data": {
    "employee_name": "AUTOQA_<run_id>",
    "department": "Development"
  },
  "missing_information": []
}
```

9.4 Structured test case example

```
{
  "name": "Create Employee",
  "type": "PRIMARY",
  "steps": [
    {"action":"NAVIGATE","path":"/employees/create"},
    {"action":"FILL","locator":{"label":"Employee Name"},"value":"AUTOQA_<run_id>"},
    {"action":"SELECT","locator":{"label":"Department"},"value":"Development"},
    {"action":"CLICK","locator":{"role":"button","name":"Save"}},
    {"action":"ASSERT_TEXT","value":"Employee created successfully"},
    {"action":"ASSERT_TABLE_ROW","value":"AUTOQA_<run_id>"},
    {"action":"SCREENSHOT","name":"employee-created"}
  ]
```

```
}
```

10. Playwright Execution and Locator Model

10.1 Currently verified core actions

Action	Purpose	State
NAVIGATE	Open an approved same-origin route	Implemented real Chromium vertical slice
FILL	Enter text in a located input	Implemented
CLICK	Click a located element	Implemented
ASSERT_VISIBLE	Verify element visibility	Implemented
ASSERT_TEXT	Verify expected text	Implemented
SCREENSHOT	Capture real viewport-sized PNG	Implemented

10.2 Required next actions

Action	Use case	Priority
SELECT	Dropdown selection	P0
CHECK / UNCHECK	Checkbox and permission options	P0
PRESS_KEY	Enter, Tab, Escape and keyboard flows	P0
WAIT_FOR_ELEMENT	Dynamic loading and AJAX	P0
ASSERT_VALUE	Input and calculated values	P0
ASSERT_URL	Redirect and navigation validation	P0
ASSERT_ENABLED / DISABLED	Validation and permission state	P0
READ_TEXT	Store actual values for later comparison	P0
ASSERT_TABLE_ROW	Verify created/updated data	P0
UPLOAD_FILE	Attachment workflows	P1
DOWNLOAD_FILE	Export/report workflows	P1
HOVER / SCROLL	Menus and long pages	P1
DATE_SELECT	Calendar controls	P1
AUTO_COMPLETE	Search suggestion controls	P1
SWITCH_TAB / FRAME_ACTION	New tabs and iframes	P2

10.3 Locator priority

```
1. Role + accessible name
2. Label
3. Placeholder
4. Stable test ID
5. Exact visible text
6. CSS fallback
7. XPath only as the last fallback
```

10.4 Execution invariants

- A step cannot be marked passed unless Playwright completed the action or assertion successfully.
- The worker must record start time, end time, duration, attempt count, actual result and error category.
- Missing locators and incorrect assertions are failures, not skipped successes.
- Every browser context is isolated by run and credential profile.
- Navigation and redirects must remain within approved origin and path rules.
- Browser crash, timeout or worker loss must produce a truthful terminal state.

11. Validation, Evidence and Result Decision

11.1 Validation layers

Layer	Checks
DOM/UI	Visibility, text, value, URL, table row, enabled/disabled state, validation message
API	Request triggered, method, status code, response fields, schema and response time
Database	Read-only query confirms UI/API values and expected state
Console	Unhandled errors, severe warnings and JavaScript exceptions
Network	Failed requests, blocked resources and server errors
Evidence integrity	Screenshot/video/trace/log exists, is readable, hashed and authorized

11.2 Evidence capture rules

Moment	Required evidence
Before main flow	Screenshot and execution metadata
After successful flow	Screenshot and final UI state
Every failure	Failure screenshot, trace, console and related network details
Main task run	Video and Playwright trace
API/DB validation	Sanitized request/response summary and query result summary
Final report	Artifact links and SHA-256 integrity metadata

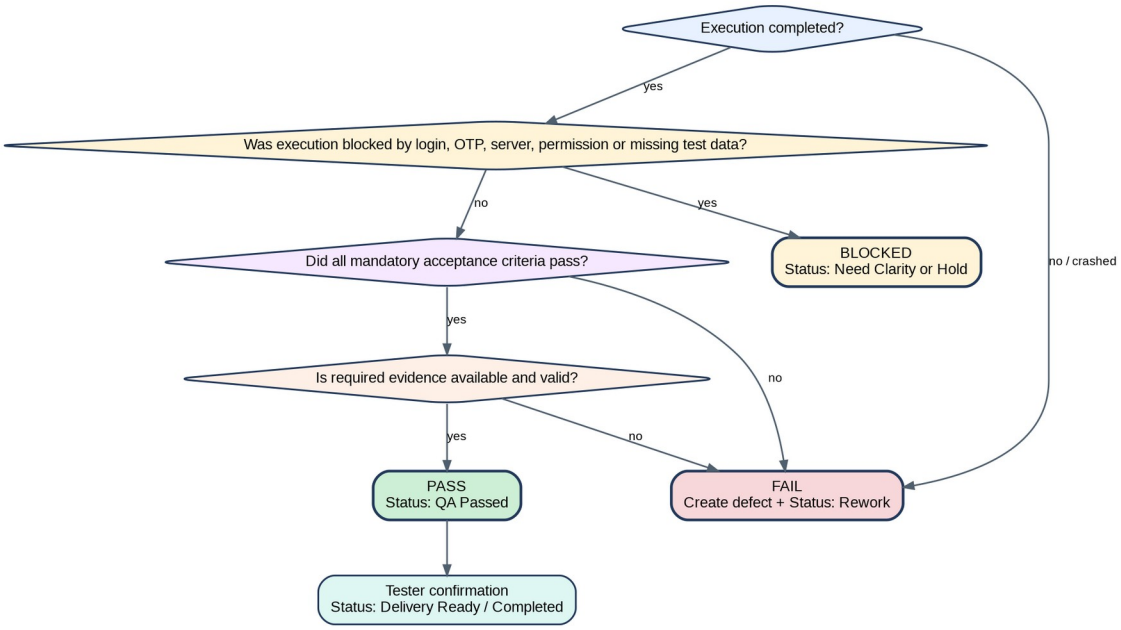


Figure 13 - Deterministic PASS, FAIL and BLOCKED decision logic

11.3 Result definitions

Result	Definition	Task outcome
PASS	All mandatory acceptance criteria passed and required evidence is valid	QA Passed, then tester confirmation for Delivery Ready/Completed
FAIL	One or more mandatory acceptance criteria failed or evidence proves incorrect behavior	Create defect and move to Rework
BLOCKED	Test could not be completed because of login, OTP, environment, permission, data or dependency issue	Need Clarity or Hold
CANCELLED	Authorized user cancelled the run	Cancelled
TIMED OUT	Execution exceeded configured limit	Failed or Blocked according to cause

12. Defects, Reports, Dashboard and Notifications

12.1 Automatic defect fields

Field	Content
Bug ID / Task ID	Traceability to source task and run
Title	Clear failure summary
Module / Feature	Affected area
Severity / Priority	AI suggestion plus tester override
Failed scenario / step	Exact test and action
Steps to reproduce	Approved executed steps
Expected / Actual	Task acceptance criterion versus observed behavior
Evidence	Screenshot, video, trace, console, network, API and DB evidence
Environment / Browser	Execution context

Field	Content
Developer / Tester	Routing and ownership
Status / Rework history	Open, Rework, Retest, Closed or Reopened

12.2 Report policy

Run type	Output
Successful run	Dashboard summary, execution history and optional lightweight JSON record
Failed run	Detailed PDF with task details, failed steps, expected vs actual, severity and evidence links
Blocked run	Dashboard and PDF/summary showing exact blocker and required action
Management view	Pass percentage, failed tasks, blocked tasks, rework, module coverage, duration and worker health

12.3 Dashboard widgets

- Testing L2 tasks waiting for review
- Plans waiting for approval
- Queued and running executions
- QA Passed, QA Failed and Blocked counts
- Delivery Ready and Rework tasks
- Open defects and latest evidence
- Pass percentage, average duration and module coverage
- Project, browser, environment and tester filters
- Worker heartbeat and queue health
- 30-day artifact retention warnings

12.4 Email notification policy

Event	Recipients	Content
Need Clarification	Superadmin, tester and task owner	Missing fields and secure task link
Execution started	Assigned tester	Task, environment and run link
PASS	Superadmin and tester	Summary and QA Passed link
FAIL	Superadmin, tester and assigned developer	Failure summary and secure report link
BLOCKED	Superadmin and tester	Blocking reason and required action
Delivery Ready	Delivery team and Superadmin	Approved result link

13. Security, Data Safety and Production Controls

13.1 PoC isolation rules

- Do not connect the real company Task Planner, production database or real employee credentials.
- Use only localhost or an isolated development network.
- Use generated demo users and non-sensitive test data.
- Keep outbound email disabled or routed to a local email sink.
- Enable audit logging for configuration, approvals, runs and artifact downloads.

13.2 Target safety controls

Control	Requirement
Domain approval	Only explicitly approved target origins
SSRF protection	Block loopback/private/link-local addresses unless explicitly configured for the local demo
DNS pinning	Resolve and pin approved addresses for the run
Same-origin policy	Block unexpected cross-origin redirects and navigation
Allowed paths	Restrict routes according to project configuration
Timeouts	Per-step and per-run limits
Downloads	Quarantine and size/type validation
Secret redaction	Mask passwords, tokens, cookies, OTP and connection strings
Credential storage	Encrypt at rest and reference by ID
Project isolation	Every query and artifact access must enforce project/tenant scope
Auditability	Record actor, approval, target, action and result

13.3 Production testing policy

Production is not the same as development

Even after permission, production should default to read-only smoke verification. Create/update/delete, negative tests, boundary tests, security payloads, email/SMS-triggering flows and financial transactions require an explicit run-level approval and cleanup plan.

Test type	Development/Staging	Production
Read-only smoke	Allowed after project approval	Allowed after run approval
Create/update test data	Allowed with cleanup policy	Disabled by default
Delete/cancel	Allowed only when task requires it	Disabled by default
Negative/boundary	Tester-approved	Separate explicit approval
Security payloads	Approved safe suite	Normally prohibited
Email/SMS/customer notification	Use test sink/suppression	Prohibited unless isolated
Database validation	Read-only	Read-only, least privilege

14. Architecture, Data Model and Recommended Folder Structure

This section defines how the platform should be separated into secure, testable services. The design keeps the temporary connectors replaceable, preserves the existing FastAPI and Playwright foundation and makes every execution, artifact and defect project-scoped.

Architecture principle

The connector, AI planner and browser executor must be separate modules. Replacing the mock task source with the approved company API should not require rewriting execution, validation, evidence or reporting.

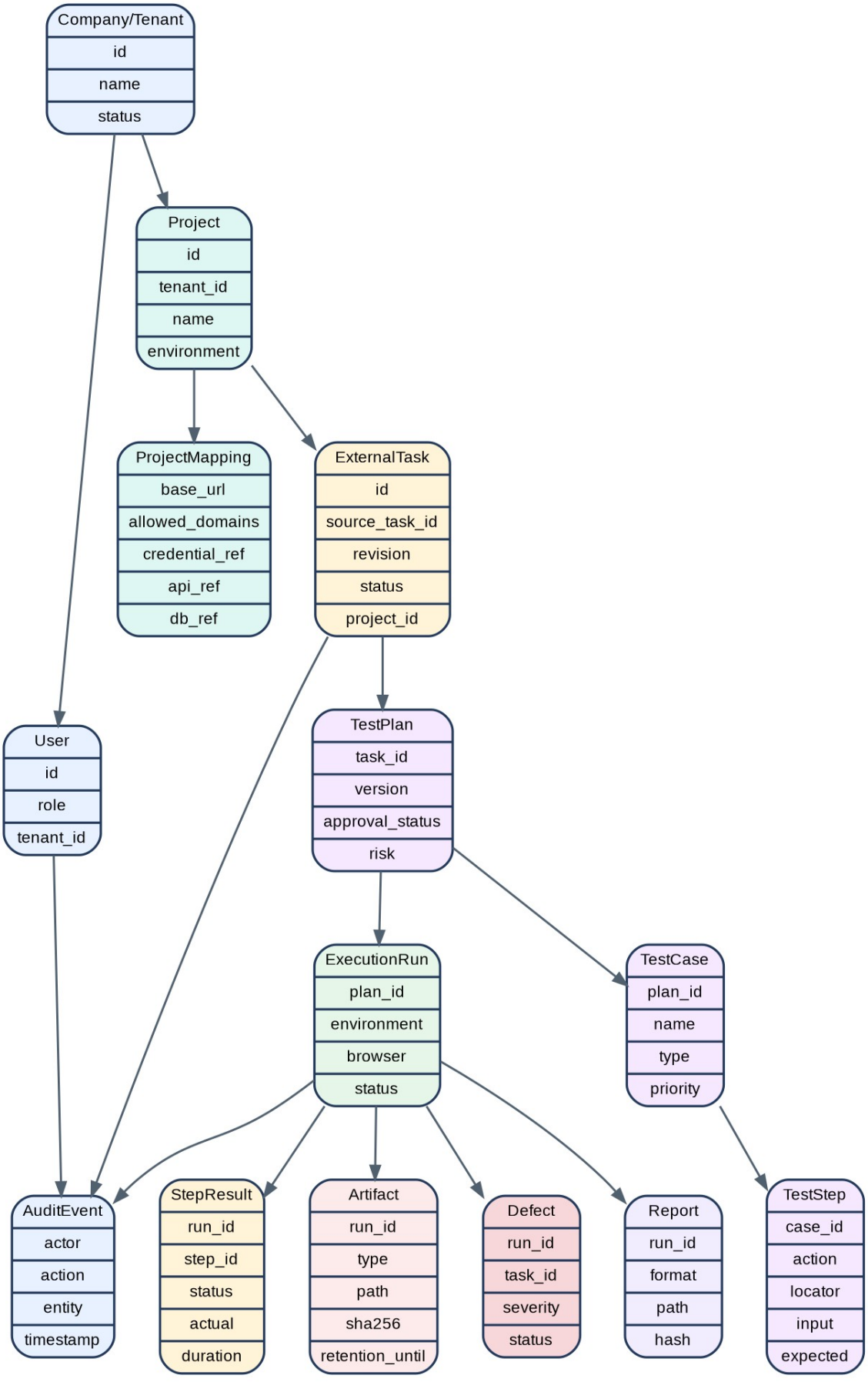


Figure 14 - Recommended core data entities and relationships

14.1 Logical architecture

```
Frontend Dashboard
    -> FastAPI API Layer
        -> Authentication / RBAC / Tenant and Project Access
        -> Mock Task Portal Connector
        -> Project and Environment Mapping
        -> AI Task Analyzer and Test Generator
        -> Approval Workflow
        -> Execution Orchestrator and Worker Queue
        -> Playwright Browser Engine
        -> Validation Engine
        -> Evidence and Artifact Service
        -> Result / Defect / Report / Notification Services
        -> Audit and Retention Services
```

14.2 Recommended folder structure

```
frontend/
  src/pages/          task queue, review, runs, defects, reports, settings
  src/components/     task cards, plan editor, evidence viewer, status chips
  src/services/       API clients

backend/
  app/api/            HTTP endpoints
  app/models/         SQLAlchemy entities
  app/schemas/        Pydantic contracts
  app/services/       business services
  app/repositories/   database access
  app/workers/        execution worker and scheduler
  app/security/       target, secrets, tenant and policy controls

agents/
  task_analyzer/
  test_generator/
  severity_classifier/
  schemas/
  prompts/

playwright/
  runner/
  actions/
  locators/
  assertions/
  network/
  evidence/

mock-task-portal/
demo-product/
fixtures/
configs/
reports/
screenshots/
videos/
traces/
logs/
exports/
tests/
```

14.3 Data retention

Data	Default retention	Notes
Run metadata	Long-term according to project policy	Small and needed for audit
Screenshots	30 days	Configurable per project
Videos	30 days	May require more storage
Traces	30 days	Sensitive; project authorization required
Console/network logs	30 days	Apply redaction
Failure PDF	30 days or defect lifetime	Secure links only
Audit events	Longer policy, e.g. 180+ days	Tamper-evident where possible

15. Technology Stack and Installation Matrix

15.1 System software

Software	Purpose	PoC	Later enterprise
Python	FastAPI, services, worker, reports and AI integration	Required	Required
Node.js LTS	React, Vite and Playwright tools	Required	Required
Git	Version control and safe review	Required	Required
Chromium / Chrome	Primary browser execution	Required	Required
Microsoft Edge	Second browser channel	Optional in first demo	Required target browser
Firefox	Third browser	After core demo	Required target browser
SQLite	Isolated PoC database	Required	Replace/retain for local only
PostgreSQL	Production-grade relational database	Not required	Recommended
Redis or equivalent	Queue, locks and scheduler support	Not required initially	Recommended
Ollama or approved LLM API	Task analysis and test generation	Required for AI demo	Approved provider
Local email sink	Safe notification preview	Recommended	Replace with approved SMTP
Docker	Reproducible runtime	Optional	Recommended

15.2 Core Python packages

Category	Recommended packages / capability
API	fastapi, uvicorn
Configuration	pydantic-settings or equivalent
Database	sqlalchemy, alembic, appropriate database driver
Automation	playwright
Testing	pytest, pytest-asyncio
Security	cryptography, secure password hashing and CSRF/session controls
Images	Pillow for screenshot validation

Category	Recommended packages / capability
Reports	Jinja2 plus a PDF generator
HTTP/API tests	httpx for API testing only, never as a substitute for browser actions
Logging	structured JSON logging and secret redaction
AI	Approved LLM client with strict JSON schema validation

15.3 Frontend and QA packages

Area	Recommended capability
Frontend	React, TypeScript, Vite and React Router
Validation	Schema-based form validation
Tables	Task, execution and defect grids with filtering
Charts	Pass/fail, trend, coverage and duration charts
QA	Playwright test runner and Trace Viewer
Accessibility later	axe-core
Performance later	Lighthouse/Web Vitals integration
OCR later	Approved OCR engine
Visual comparison later	Pixel/structural comparison with thresholds

15.4 Environment variables

Variable group	Examples	Rule
Application	APP_ENV, API_BASE_URL, FRONTEND_URL	No secrets in source code
Database	DATABASE_URL	Encrypted/secret-managed in enterprise
Security	SESSION_SECRET, CSRF_SECRET, ENCRYPTION_KEY	Mandatory rotation policy
Playwright	BROWSER_CHANNEL, HEADLESS, RUN_TIMEOUT	Project/run overrides within policy

15.4 Environment variables (continued)

Variable group	Examples	Rule
Artifacts	ARTIFACT_ROOT, RETENTION_DAYS, MAX_FILE_SIZE	Containment checks required
AI	AI_PROVIDER, AI_MODEL, AI_ENDPOINT	No prompt output executes directly
Email	SMTP_HOST, SMTP_USER, SMTP_SECRET	Use local sink in PoC
Task connector	TASK_PORTAL_BASE_URL, CONNECTOR_TOKEN	Mock endpoint initially

15.5 PoC installation readiness checklist

Check	Expected result
Python environment	Backend dependencies install and import successfully
Node environment	Frontend and QA TypeScript checks pass
Playwright browser	Chromium launches and executes the six verified actions
Database	Temporary database migrates and starts cleanly
Mock Task Portal	Testing L2 API returns a demo task
Demo Product	Login and Employee form are reachable locally
Artifact directories	Screenshots, videos, traces and reports are writable and contained
Secrets	No real company credential is present in source, logs or demo data

16. APIs and Integration Contracts

16.1 Mock Task Portal APIs

```
POST  /api/tasks
GET   /api/tasks?status=TESTING_L2
GET   /api/tasks/{task_id}
PATCH /api/tasks/{task_id}
PATCH /api/tasks/{task_id}/status
POST  /api/tasks/{task_id}/developer-notes
POST  /api/tasks/{task_id}/qa-result
```

16.2 AutoQA APIs

```
POST /api/task-imports/sync
GET  /api/task-imports?status=WAITING_REVIEW
POST /api/task-imports/{id}/analyze
POST /api/test-plans/{id}/generate
PATCH /api/test-plans/{id}
POST /api/test-plans/{id}/approve
POST /api/runs
POST /api/runs/{id}/approve
POST /api/runs/{id}/cancel
GET  /api/runs/{id}
GET  /api/runs/{id}/steps
GET  /api/runs/{id}/artifacts
GET  /api/defects
GET  /api/reports/{run_id}/failure.pdf
```

16.3 Connector result payload example

```
{
  "source_task_id": "24125",
  "source_revision": 3,
  "autoqa_run_id": "RUN-20260717-001",
  "qa_result": "FAIL",
  "task_status": "REWORK",
  "summary": "Remaining quantity did not match expected calculation",
  "severity": "HIGH",
  "report_url": "/secure/reports/RUN-20260717-001/failure.pdf",
  "completed_at": "2026-07-17T12:30:00+05:30"
}
```

16.4 Duplicate and revision handling

Idempotency key

Use source_system + source_task_id + source_revision as the import key. A revised task creates a new revision record; it must not overwrite the evidence and history of an earlier run.

17. Complete Phase Plan



Figure 15 - High-level roadmap from foundation to approved company pilot

Phase	Name	Main work	Priority	Exit output
0	Audit and Backup	Verify current code, tests, DB, security and create safe branch	P0	Completed baseline
1	Mock Task Portal	Local task planner, roles, fields and statuses	P0	Testing L2 task source
2	Demo Product Website	ERP-like safe target with configurable pass/fail behavior	P0	Repeatable test target
3	Task Connector	Import Testing L2 tasks and sync results	P0	Task-to-AutoQA integration
4	Project Mapping	URL, paths, credentials, API/DB and cleanup policies	P0	Safe target configuration
5	AI Task Analyzer	Module, scope, risk, data and missing information	P0	Structured task understanding
6	AI Test Generator	Primary, negative and optional regression tests	P0	Reviewable structured cases
7	Tester Approval	Plan editor, approval, frozen version and audit	P0	Human-controlled execution
8	Execution Orchestrator	Queue, atomic claim, timeout, cancellation and recovery	P0	Reliable run lifecycle
9	Full Playwright Actions	Add select, check, wait, value/url/table assertions	P0	Useful ERP browser automation
10	Login and OTP	Credential profiles and manual OTP checkpoint	P0	Real authenticated flows
11	Validation Engine	UI, API, DB, console and network checks	P0/P1	Trustworthy result
12	Evidence Engine	Screenshots, video, trace, logs, hash and retention	P0	Auditable proof
13	Result Engine	PASS, FAIL, BLOCKED and status mapping	P0	Consistent decisions
14	Defect Creation	Evidence-backed defect and rework history	P0	Developer-ready failure
15	Dashboard and PDF	Management dashboard and failure PDF	P0	Visible output
16	Email Notification	Secure result notifications	P1	Operational delivery

Phase	Name	Main work	Priority	Exit output
17	Advanced Testing	Responsive, accessibility, performance, security, OCR	P2	Enterprise expansion
18	Team-Lead Demo	Run one PASS and one FAIL scenario end to end	P0	Approval evidence
19	Company Pilot	Approved read-only task connector and development URL	Future	Controlled real integration

17.1 Phase dependency and size matrix

Phase group	Dependency	Indicative size	Do not proceed until
0 Foundation	None	S	Baseline tests and backups are verified
1-2 Safe PoC systems	Foundation	M	Mock task and demo product work independently
3-4 Integration/config	PoC systems	M	Task and target mapping are deterministic
5-7 AI and approval	Structured task data	L	Schema validation and manual approval exist
8-10 Execution	Approved plans and target config	L	Queue integrity and browser isolation are verified
11-16 Assurance/output	Real execution	L	Results and evidence are trustworthy
17 Advanced	Core PoC complete	XL	Team lead approves scope
18 Demo	Core P0 complete	S	Full pass/fail scripts are repeatable
19 Company pilot	Written approval	L	Security and access reviews are complete

17.2 Detailed exit criteria by phase

Phase 0 - Audit and Backup

- Clean git state or recorded diff
- Database backup and migration check
- Existing tests and current Playwright vertical slice pass
- No dummy action can be counted as complete

Phase 1 - Mock Task Portal

- Create/assign/view/update task
- Developer can move assigned task to Testing L2
- Status history and role permissions work
- Testing L2 API returns correct tasks

Phase 2 - Demo Product

- Login works
- Employee form has textbox/dropdown/save/table
- One configurable bug can be enabled
- All data is local and disposable

Phase 3 - Connector

- Imports only Testing L2
- Prevents duplicates
- Stores source revision
- Can sync QA status safely

Phase 4 - Mapping

- Project automatically resolves correct demo URL
- Missing mapping blocks execution
- Credentials are referenced securely
- Cross-origin path is blocked

Phase 5 - Analyzer

- Returns schema-valid scope
- Identifies missing details
- Does not invent unrelated modules
- Manual fallback works

Phase 6 - Generator

- Generates supported actions only
- Primary/negative/regression are separated
- Draft is editable
- Approved version is immutable

Phase 7 - Approval

- Only authorized tester/superadmin can approve
- Approval audit is recorded
- Production has extra confirmation
- Developer cannot self-approve QA

Phase 8 - Orchestrator

- Atomic worker claim
- Cancellation remains terminal
- Stale worker recovery works
- Browser is always cleaned up

Phase 9 - Playwright actions

- Dropdown, checkbox, wait and assertions work
- Missing locator fails
- Every step records actual result
- No false pass

Phase 10 - Authentication

- Username/password profile works
- OTP pause/resume works
- OTP is never stored
- Two tester sessions remain isolated

Phase 11 - Validation

- UI result is deterministic
- API and DB are optional configured layers
- Console/network failures are visible
- AI does not decide pass/fail

Phase 12 - Evidence

- Real screenshot dimensions
- Video and trace open successfully
- Artifacts are hashed and project-scoped
- Retention cleanup works

Phase 13 - Results

- PASS/FAIL/BLOCKED rules tested
- Status mapping correct
- PASS requires evidence
- Delivery requires confirmation

Phase 14 - Defects

- Failure creates complete defect
- Duplicate detection exists
- Retest history is retained
- Tester can override severity

Phase 15 - Reports

- Failure PDF is readable and complete
- Dashboard filters work
- Other project users cannot access report
- Downloads are audited

Phase 16 - Email

- Uses secure links
- No secrets in email
- Correct recipients

- Delivery failures are recorded

Phase 18 - Demo

- One PASS task from creation to Delivery Ready
- One FAIL task from Testing L2 to Rework
- Evidence shown live
- No company system connection

Phase 19 - Pilot

- Written approval
- Read-only task API
- Approved development URL and test account
- Rollback and stop controls

18. MVP Demo Scenarios and Acceptance Criteria

18.1 PASS demonstration

```
1. Superadmin creates DEMO-001 in Mock Task Portal.
2. Developer completes the Employee form and moves it to Testing L2.
3. AutoQA imports the task and generates a test plan.
4. Tester approves primary and negative scenarios.
5. Playwright logs in to Demo Product.
6. Employee name is filled; Department is selected; Save is clicked.
7. Success message and employee table row are verified.
8. Before/after screenshots, video and trace are stored.
9. Result is PASS and task becomes QA Passed.
10. Tester confirms and task becomes Delivery Ready/Completed.
```

18.2 FAIL demonstration

```
1. Enable the Demo Product intentional Save-button defect.
2. Repeat the same approved task flow.
3. The button action or expected result fails.
4. AutoQA captures a failure screenshot, trace, video, console/network evidence.
5. Result is FAIL.
6. AutoQA creates a defect with severity, steps, expected and actual behavior.
7. Mock Task Portal status becomes Rework.
8. Developer sees the evidence and can submit a new revision for retest.
```

18.3 BLOCKED demonstration

```
1. Disable the demo test account or require a missing OTP.
2. AutoQA pauses or fails readiness validation.
3. No product defect is created.
4. Result is BLOCKED with reason "Authentication unavailable".
5. Task becomes Need Clarity or Hold.
```

18.4 MVP acceptance checklist

Area	Acceptance criterion
Task source	Testing L2 task is imported exactly once

Area	Acceptance criterion
AI plan	Plan matches the task and contains no unrelated module
Approval	No execution starts without tester/superadmin approval
Browser	Real Chromium opens the target and performs supported actions
Form control	Textbox, dropdown and Save interaction work
Validation	Success message and data row are verified
Evidence	Before, after, success/failure screenshot, video and trace exist
Result	PASS/FAIL/BLOCKED is deterministic
Defect	Failure creates evidence-backed defect
Status sync	Pass, rework and blocked statuses map correctly
Reports	Failure PDF is generated and authorized
Isolation	Demo uses no company data or credentials

19. Team-Lead Approval and Real Company Integration

19.1 Team-lead demo package

- This project concept PDF
- Live local Mock Task Portal
- Live local Demo Product Website
- One PASS and one FAIL recorded demo
- Failure PDF, screenshots, video and trace
- Security statement confirming no company system was connected
- Pilot access request describing exact read-only APIs and test accounts needed

19.2 Approval questions

Decision	Required answer
Task Planner access	Is a read-only API available for Testing L2 tasks?
Status update	Can AutoQA update status through a controlled API, or should a tester update manually?
Development target	Which development/staging URL is approved?
Test account	Which isolated role account can be used?
OTP	How should first-time OTP be handled?
API documentation	Which Swagger/Postman specification is approved?
Database validation	Is read-only database access approved, and what DB type?
Data cleanup	Should created test data be kept, cancelled, deleted or reviewed by tester?
Production	Which smoke checks are allowed and which are prohibited?
Retention	Is 30-day evidence retention approved?

19.3 Controlled pilot sequence

7. Obtain written approval and named owners.
8. Create an approved test account and least-privilege connector credential.
9. Connect only the Testing L2 read endpoint first.

10. Map one non-critical project to an approved development URL.
11. Import one pilot task and review the generated plan manually.
12. Run a non-destructive task flow with full evidence.
13. Review security, performance and false-positive/false-negative behavior.
14. Approve limited rollout or return to PoC for corrections.

Connector replacement strategy

The PoC must keep a clean connector interface. After approval, replace MockTaskConnector with CompanyTaskPlannerConnector and replace DemoProjectMapping with the approved development mapping. The planner, approval, executor, evidence and report modules should not require a rewrite.

20. Current Project State and Remaining Work

20.1 Existing foundation that can be reused

- FastAPI backend, authentication, RBAC and project-scoped access foundation
- Manual test management, lifecycle and execution history
- Connection target governance and safe target controls
- Defect management, audit logging and basic reports
- Real Chromium execution vertical slice for NAVIGATE, FILL, CLICK, ASSERT_VISIBLE, ASSERT_TEXT and SCREENSHOT
- Real screenshot validation and cross-project automated-run authorization fix

20.2 Major remaining product work

Area	Current direction	Required next state
Task workflow	No temporary company-like task source	Build Mock Task Portal and connector
Demo target	No complete safe ERP demo	Build Demo Product with pass/fail toggles
AI	Prompts/concept not complete end to end	Structured analyzer and generator with schema validation
Actions	Six core actions	Add SELECT, CHECK, WAIT, value/url/table assertions
Approval UX	Existing lifecycle foundation	Task review and generated-plan approval workspace
Evidence	Real screenshot; trace support foundation	Video, standardized before/after/failure capture and viewer
Validation	Basic DOM assertions	Task-related API/DB/console/network validation
Status sync	No mock task synchronization	PASS/FAIL/BLOCKED mapping and connector callback
Reports	Basic report data	Detailed failure PDF and dashboard workflow
Notifications	Missing	Dashboard and email
Advanced QA	Not first PoC	Responsive, accessibility, performance and safe security checks

20.3 Recommended immediate next development task

Next task

Build the local Mock Task Portal and the local Demo Product Website as two isolated applications, then connect one Testing L2 task to the existing real Playwright vertical slice. Do not start company integration or

advanced AI features until this basic task-to-browser-to-result demonstration works.

21. Risks, Mitigations and Governance

Risk	Impact	Mitigation
AI creates an incorrect flow	False failure or unsafe action	Schema validation, allowlisted actions and mandatory manual approval
Task description is incomplete	Wrong scope	Need Clarity status and required-field gate
UI changes break locators	Execution instability	Accessible locators, fallback order and reviewed self-healing suggestions
False PASS	Loss of trust	No-op actions prohibited; deterministic assertions and required evidence
Production data mutation	Business impact	Development-first policy, run-level destructive approval and cleanup plan
Credential leakage	Security incident	Encryption, redaction, secret references and access audit
Cross-company data access	Critical isolation breach	Tenant/project filter on every API, job and artifact
Duplicate task import	Duplicate testing and defects	Source ID + revision idempotency key
Worker concurrency race	Duplicate execution	Atomic claim/lease and idempotent state transitions
Large evidence storage	Cost and performance	30-day retention, quotas and compression
Company portal UI changes	Connector failure	Use API integration; UI scraping only as temporary fallback
No API/DB access	Incomplete validation	Run UI-only with transparent coverage label, not fake completeness

21.1 Governance rules

- One task at a time; audit before modification; minimal safe patches; preserve working architecture.
- No dummy testing, fake completion or hardcoded PASS results.
- Every completed feature must have code evidence and passing tests.
- Every production-affecting capability requires named approval and an auditable policy.
- Security findings in AutoQA itself are P0 and must be fixed before expanding automation scope.

Appendix A - Sample Task Record

```
Task ID: DEMO-001
Project: Demo ERP
Module: Employee
Task Title: Employee form Save button functionality
Task Description: Create a new employee and show the saved employee in the result table.
Developer: Demo Developer
Tester: Tester 1
Target Date: 20-07-2026
Environment: Development
Application URL: http://localhost:<demo-port>
Page Path: /employees/create
Expected Result: Employee is saved and a success message is displayed.
Acceptance Criteria:
  - Name is mandatory
  - Department dropdown works
  - Save button creates the employee
  - Created employee appears in the table
Test Data: AUTOQA_<run_id>, Department=Development
Status: Testing L2
```

Appendix B - Sample Failure Report Layout

Section	Contents
Header	Project, task ID, run ID, environment, browser, date and tester
Summary	PASS/FAIL/BLOCKED, duration, completed scenarios and failed scenarios
Failure	Bug title, module, severity and priority
Reproduction	Approved steps and failed step
Expected vs Actual	Task acceptance criterion versus observed result
Evidence	Before/failure screenshots, video, trace, console, network, API and DB links
Status action	Rework / Need Clarity / Hold
Audit	Plan version, approver and timestamps

Appendix C - Initial Test Data Rules

- Use a unique AUTOQA_<run_id> prefix for created records.
- Never use real customer names, phone numbers, invoices or employee identifiers in the PoC.
- Store generated data values with the run for reproducibility.
- Define keep/cancel/delete cleanup behavior per project and per test case.
- Do not run duplicate, maximum-length or destructive scenarios in production without explicit approval.

Appendix D - Definition of Done

- The feature is implemented in the actual code path, not only in UI text or prompts.
- Relevant unit, integration and Playwright tests pass.
- Failure behavior and authorization are tested.
- Evidence is genuine, readable and project-scoped.
- Documentation and configuration examples are updated.
- No existing architecture or security control is weakened.

- The final result is verified using a real end-to-end workflow.

Final Project Concept

Final statement

iFive AutoQA X will first be demonstrated as a fully isolated local proof-of-concept. A developer task enters Testing L2 in a mock task portal. AutoQA reads the exact task, generates structured task-specific tests, waits for Superadmin/Tester approval, executes the real flow against a demo product using Playwright, validates the result, captures complete evidence and returns PASS, FAIL or BLOCKED. After a successful team-lead demo and written permission, only the connector and target mapping are replaced with approved company development systems.

End of document